** Write your name etc.

CSC 583 HW#2-1: Vector Embeddings and RAG

Vector Embeddings and RAG Starter Code Multi-Document RAG System for AI Policy Analysis

```
# -*- coding: utf-8 -*-
# Install libraries and tools
# ----------------------------
!pip install -qU requests==2.32.4 chromadb langchain langchain-chroma langchain-huggingface langchain_openai langchain_community sentence-transfor
```

```
# Imports
# ----------------------------
import os, re, shutil
import requests
import chromadb
from typing import List, Dict, Tuple
from IPython.display import Markdown, display

# LangChain imports
from langchain.schema import Document
from langchain.document_loaders import PyPDFLoader, TextLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import Chroma
from langchain_huggingface import HuggingFaceEmbeddings
from langchain.chains import LLMChain
from langchain_openai import ChatOpenAI, OpenAI
from langchain.prompts import PromptTemplate
```

Part 1: Define Core Functions

```
# 1-1: Load and preprocess documents
# ----------------------------
def load_documents() -> List[Document]:
    """
    Load the four AI policy documents from their URLs.
    Returns a list of Document objects.
    """
    # (1) TODO: Implement this function
    # Load all four documents using appropriate LangChain document loaders
    # Handle both PDF and text documents
    # Return a list of Document objects
    url = "https://raw.githubusercontent.com/ntomuro/CSC583_2025Fall/main/HW2-1/data"
    document_filenames = [
        "Blueprint-for-an-AI-Bill-of-Rights.pdf", # The White House "Blueprint for an AI Bill of Rights" (2022)
        "2023-24283.pdf", # Executive Order on the Safe, Secure, and Trustworthy Development and Use of Artificial Intelligence (2023)
        "NIST.AI.100-1.pdf", # NIST AI Risk Management Framework (AI RMF 1.0) Overview
        "cellar_e0649735-a372-11eb-9585-01aa75ed71a1.0001.02_DOC_1.pdf" # The EU Artificial Intelligence Act (Key Principles Summary)
    ]
    ## Continue the TODO and collect raw documents in a variable 'documents'
    ## and return it.  You'll use langchain's PyPDFLoader, TextLoader to do.




    return documents

def preprocess_documents(documents: List[Document]) -> List[Document]:
    """
    Split documents into chunks for vector storage.
    """
    # (2) TODO: Implement text splitting
    # Use RecursiveCharacterTextSplitter with appropriate (default) parameters.
    # Consider what chunk_size and chunk_overlap work best for multi-document retrieval
    # Return a list of document chunks from 'documents'.



    # for now; change to something appropritate based on your code
    return []


# 1-2: Vector Store Population
# ----------------------------
```

```
def create_vector_store(documents: List[Document], embedding_model_name: str, client):
    """
    Create and return a retriever from documents using the specified embedding model.
    """
    # (3) TODO: Implement vector store creation
    # Initialize embedding function with the specified model.
    # Create Chroma vector store from documents.
    # Return a 'retriever' with search_kwargs={"k": 4}.




    return retriever

# 1-3: Query & Evaluation Set
# ----------------------------
def create_test_queries() -> List[str]:
    """
    Create a diverse set of 5 test queries as specified in the assignment.
    """
    # (4) TODO: Implement this function to return 5 well-designed test queries
    # Include:
    #   Single-Document Factual
    #   1 Multi-Document "Search & Combine"
    #   1 Thematic Synthesis
    #   1 Comparative
    #   1 Specific Definition/Scope
    # You can hard-code your own queries, or automatically generate queries in some way.

    queries = [
        # "Single-document factual question here",
        # "Multi-document search and combine question here",
        # "Thematic synthesis question here",
        # "Comparative question here",
        # "Specific definition/scope question here"
    ]
    return queries

# 1-4: The RAG Chain
# ----------------------------

def run_rag_query(query: str, retriever) -> Tuple[str, List[Document]]:
    """
    Execute a RAG query: retrieve relevant context and generate an answer.
    Returns the generated answer and the retrieved documents for evaluation.
    """
    # (5) TODO: Implement the RAG query execution
    # Retrieve relevant chunks using the retriever
    # Construct a high-quality prompt that includes context and query
    # Use an LLM chain ("gpt-3.5-turbo", with 'temperature=0') to generate the answer
    # Assign the answer and retrieved documents to variables 'answer' and
    # 'retrieved_docs', and return them.









    return answer, retrieved_docs

# 1-5: Evaluation
# ----------------------------
def evaluate_results(query: str, retrieved_docs: List[Document], answer: str) -> Tuple[int, int]:
    """
    Manually evaluate the retrieval quality and answer quality.
    Returns (retrieval_score, answer_score) on scale 1-3.
    """
    # Manually evaluate and return scores
    # Retrieval Score: 1=Irrelevant, 2=Partially relevant, 3=Highly relevant
    # Answer Score: 1=Incorrect, 2=Partially correct, 3=Correct and comprehensive
    # Code is written.  You only enter your manual evaluation scores in real-time.
    print(f"Query: {query}")
    print(f"Retrieved {len(retrieved_docs)} documents")
    print(f"Answer: {answer}")
    print("---")

    retrieval_score = int(input("Retrieval score (1-3): "))
    answer_score = int(input("Answer score (1-3): "))

    return retrieval_score, answer_score
```

Part 2: Experiment (main())

This cell is mostly complete except for your OpenAI key and HuggingFace token. You can make slight modifications to do more experiments, but do not make large changes.

```
# 2: The Main Experiment
# ---------------------------
def main():
    """
    Main function to run the complete RAG experiment.
    """
    def safe_name(s):
        """
        Convert a string to a valid file name by removing invalid characters
        and using underscores to separate words.
        """
        return re.sub(r'[^0-9A-Za-z._-]', '_', s)

    # (5) TODO: Set your OpenAI API key and HuggingFace token.
    # You can do in any way you like.
    os.environ["OPENAI_API_KEY"] = "..."  # TODO: Replace with your key
    os.environ["HUGGINGFACE_HUB_TOKEN"] = "..."

    # Define embedding models to test
    embedding_models = [
        "all-MiniLM-L6-v2",  # 384
        "all-mpnet-base-v2", # 768
        "multi-qa-MiniLM-L6-cos-v1" # 384
    ]

    # Load and preprocess documents
    print("Loading documents...")
    raw_documents = load_documents()
    print(f"Loaded {len(raw_documents)} raw documents")

    print("Preprocessing documents...")
    processed_documents = preprocess_documents(raw_documents)
    print(f"Created {len(processed_documents)} document chunks")

    # Create test queries
    test_queries = create_test_queries()
    print(f"Created {len(test_queries)} test queries")

    # Store results for analysis
    results = {}

    # Run experiment for each embedding model
    for model_name in embedding_models:
        print(f"\n{'='*50}")
        print(f"Testing model: {model_name}")
        print(f"{'='*50}")

        # Create a unique DB for the embedding model
        model_key = safe_name(model_name)
        db_path = f"./chroma_db_{model_key}"   # unique per model

        chroma_client = chromadb.PersistentClient(path=db_path)

        # Create vector store with current model
        retriever = create_vector_store(processed_documents, model_name, chroma_client)

        model_results = {
            'retrieval_scores': [],
            'answer_scores': [],
            'details': []
        }

        # Test each query
        for i, query in enumerate(test_queries): ####
            print(f"\nQuery {i+1}: {query}")

            # Run RAG query
            answer, retrieved_docs = run_rag_query(query, retriever)

            # Evaluate results
            retrieval_score, answer_score = evaluate_results(query, retrieved_docs, answer)

            # Store scores
            model_results['retrieval_scores'].append(retrieval_score)
            model_results['answer_scores'].append(answer_score)
```

```
            model_results['details'].append({
                'query': query,
                'answer': answer,
                'retrieved_docs': retrieved_docs
            })

        # Calculate averages
        model_results['avg_retrieval'] = sum(model_results['retrieval_scores']) / len(model_results['retrieval_scores'])
        model_results['avg_answer'] = sum(model_results['answer_scores']) / len(model_results['answer_scores'])

        results[model_name] = model_results

        print(f"\nModel {model_name} - Avg Retrieval: {model_results['avg_retrieval']:.2f}, Avg Answer: {model_results['avg_answer']:.2f}")

    return results, processed_documents, test_queries
```

Part 3: Run Experiment

```
# Execute the experiment
final_results, documents, queries = main()
print (final_results)
```

Print Results -- Run this cell after you finish writing your code. Report the results table in your write-up!

```
# Analysis Helper Functions
# ----------------------------

def print_results_table(results: Dict):
    """Print a formatted results table for the report."""
    print("\n" + "="*60)
    print("EXPERIMENT RESULTS SUMMARY")
    print("="*60)
    print(f"{'Model':<25} {'Avg Retrieval':<15} {'Avg Answer':<15}")
    print("-"*60)

    for model_name, model_results in results.items():
        print(f"{model_name:<25} {model_results['avg_retrieval']:<15.2f} {model_results['avg_answer']:<15.2f}")

def compare_retrieval_for_query(results: Dict, query_index: int):
    """Compare retrieval performance for a specific query across models."""
    print(f"\nRetrieval comparison for Query {query_index + 1}:")
    for model_name, model_results in results.items():
        retrieval_score = model_results['retrieval_scores'][query_index]
        answer_score = model_results['answer_scores'][query_index]
        print(f"  {model_name}: Retrieval={retrieval_score}, Answer={answer_score}")

# Print the final results
# ----------------------------
print_results_table(final_results)
for query in queries:
  compare_retrieval_for_query(final_results, queries.index(query))
```